SAFYRON ENGINEERING

Design and Implementation of a Intelligent E-Learning Platform

AI-Assisted Quiz Generation and Instructor Validation

Nature	One-month professional internship report
Field of study	Software engineering
Student	Ismail SALEM
Host organization	SAFYRON ENGINEERING
Professional supervisor	Ashref Ammar
Academic supervisor	EPI Digital School
Academic year	2026–2027

Angular / Spring Boot / MySQL platform for compliance training, learning progress, controlled assessment, and certification.

Report version: November 11, 2026

Acknowledgements

I would like to express my sincere gratitude to **Mr. Ammar Achraf** for his warm welcome, constant availability, professional guidance, and the trust he placed in me throughout this internship. His support enabled me to gain a deeper understanding of the company's environment and helped me approach my work in a structured and organized manner.

I would also like to thank **SAFYRON ENGINEERING** for providing me with the opportunity to discover its professional environment and gain valuable insight into the challenges related to the processing and monitoring of technical requests.

Finally, I would like to extend my appreciation to **EPI Digital School of Sousse** for the software engineering education and the academic framework provided for this professional internship. This experience allowed me to put my acquired knowledge into practice and carry out a complete development process, from requirements analysis to the validation of the proposed solution.

Abstract

This report presents the design and implementation of Safyron Learning, a web learning platform for organizing training modules, teaching materials and multiple-choice assessments. The project was completed during a one-month professional internship. Its central objective was to provide a controlled workflow from module publication to instructor approval of a learner's assessment.

The solution combines Angular 21, Spring Boot 3.3, MySQL persistence, document extraction strategies and Google's OpenAI-compatible Gemini API for AI-assisted quiz generation. Instructors can create a quiz manually or generate a draft from uploaded material, review and publish it, then inspect the learner's answers question by question. A quiz is submitted only once and the final certification state depends on instructor approval. JWT-based authentication is integrated in the application security design and is available as the authentication layer for the deployed project.

The report explains the context, requirements, architecture, data model, implementation choices and validation process. It also discusses deletion consistency, Swagger testing, user experience, limitations and future improvements.

Keywords: e-learning; Angular; Spring Boot; MySQL; quiz; artificial intelligence; JWT; document extraction; instructor approval.

Contents

Acknowledgements i

Abstract ii

List of Acronyms 1

General Introduction 2

1 Project Context and Scope 3
- 1.1 Host Organization and Internship Framework . . . 3
- 1.2 Project overview . . . 3
- 1.3 Initial problem . . . 4
- 1.4 Stakeholders . . . 4
  - 1.4.1 Learner . . . 4
  - 1.4.2 Instructor . . . 4
  - 1.4.3 Administrator . . . 4
- 1.5 Scope and objectives . . . 5
- 1.6 Planning . . . 5
- 1.7 Project boundaries . . . 5

2 Analysis and Design 7
- 2.1 Functional requirements . . . 7
- 2.2 Actors and principal use cases . . . 8
  - 2.2.1 Non-functional requirements . . . 8
- 2.3 Use-case model . . . 9
  - 2.3.1 Module and material scenario . . . 9
  - 2.3.2 Quiz authoring and publication scenario . . . 10
  - 2.3.3 Learner submission scenario . . . 10
  - 2.3.4 Instructor approval scenario . . . 10
  - 2.3.5 Textual description of major use cases . . . 10
- 2.4 User-interface conception . . . 10
- 2.5 System architecture . . . 11
- 2.6 Technical conception . . . 11
- 2.7 Design principles and patterns . . . 11
- 2.8 Security design . . . 12
- 2.9 Data model and integrity . . . 12
- 2.10 Quiz lifecycle . . . 14
- 2.11 AI design . . . 15

3 Implementation and Validation **18**
3.1 Development environment and technologies 18
3.2 Backend implementation 18
3.3 Frontend implementation 19
3.4 Learner experience 20
3.5 Instructor studio 22
3.6 Administrator console 23
3.7 Answer persistence and grading 23
3.8 File management and deletion 24
3.9 Swagger and API usage 24
3.10 Implementation difficulties and corrective actions 24
3.11 Validation and Results 24
3.12 Validation strategy 24
3.13 Validation traceability 25
3.14 Functional scenarios 25
3.15 Quality results 25
3.16 Security and operational limitations 26
3.17 Difficulties, limitations and perspectives 26
3.18 User experience observations 26
3.19 Personal contribution 26

General Conclusion **27**

References and Webography **28**

A Project Structure and Reproduction **29**
A.1 Directory structure 29
A.2 Representative REST contracts 29
A.3 Diagram catalogue 30

List of Figures

1.1 Safyron Engineering visual identity 3

2.1 Global use-case diagram 9
2.2 Component architecture 11
2.3 Entity–repository–service responsibilities 11
2.4 Authentication sequence 12
2.5 Quiz validation and lifecycle classes 13
2.6 Learner takes and submits a quiz 14
2.7 Detailed AI quiz-generation use case 15
2.8 AI quiz-generation sequence 16
2.9 AI client and generation classes 17
2.10 Document extraction classes 17

3.1 Live screenshot captured from the running login page 20
3.2 Live screenshot captured from the learner portal 21
3.3 Live screenshot captured from the instructor studio 22
3.4 Live screenshot captured from the administrator console 23

List of Tables

1.1 Project objectives 5
1.2 Simplified Gantt diagram – provisional internship planning 5

2.1 Functional requirements matrix 7
2.2 Actors and responsibilities 8
2.3 Non-functional requirements and acceptance evidence 8
2.4 Main use-case scenarios 10
2.5 Security responsibilities 12
2.6 Persistent entity dictionary 13

3.1 Technologies and versions 18
3.2 Main backend layers 19
3.3 Validation activities 25
3.4 Traceability between requirements and validation 25

A.1 Project diagrams stored with the report 30

List of Acronyms

API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CORS	Cross-Origin Resource Sharing
DIP	Dependency Inversion Principle
DTO	Data Transfer Object
E2E	End-to-End
HTTP	HyperText Transfer Protocol
IA	Artificial Intelligence
JPA	Jakarta Persistence API
JSON	JavaScript Object Notation
JWT	JSON Web Token
LMS	Learning Management System
MCQ	Multiple Choice Question
RBAC	Role-Based Access Control
REST	Representational State Transfer
SRP	Single Responsibility Principle
SPA	Single Page Application
SQL	Structured Query Language
UML	Unified Modeling Language

General Introduction

Digital learning has become an important component of professional training, particularly in environments where compliance, safety and traceability matter. An effective learning platform must do more than display documents: it must connect the training material, assessment, learner progress and validation decision in one coherent workflow.

The Safyron Learning project addresses this need through a role-oriented web application. Learners access published modules and their supporting files. Instructors create the modules, prepare assessments and review submissions. Administrators oversee users, modules and global activity. The platform also uses artificial intelligence to reduce the time required to draft questions from source documents, while preserving human review before publication.

The internship was therefore both a development assignment and an exercise in software quality. Several business rules had to remain consistent across the Angular interface, REST API and relational database. A deleted module must not leave orphaned quizzes or files; an answer must remain associated with the correct question; and a learner must not obtain final certification before the instructor has reviewed the submission.

This report follows the structure of the project. Chapter 1 presents the context, problem statement, stakeholders and scope. Chapter 2 formalizes the requirements and explains the architecture and design. Chapter 3 presents the implementation, validation results, live website interface and technical limitations. The conclusion summarizes the contribution and proposes future work.

Chapter 1

Project Context and Scope

Introduction

This chapter presents the host organization, the internship framework, the existing training context and the scope selected for the project. It explains why Safyron Learning is relevant to the activities of Safyron Engineering and how the one-month work was organized.

1.1 Host Organization and Internship Framework

The professional internship took place from 1 to 31 August 2026 at **Safyron Engineering**, located in Rue Lac Toba, Les Berges du Lac, Tunis. The company operates in the field of industrial technical studies, including fire protection, hazard studies and industrial risk analysis. These activities require accurate technical documentation and continuous professional training.



Figure 1.1: Safyron Engineering visual identity

The internship was completed as part of the fifth year of the software engineering program at EPI Digital School Sousse. The work was carried out individually and supervised professionally by Mr Ammar Achraf, founder of Safyron Engineering. The company context made it possible to connect software engineering practices with the operational need for reliable safety and compliance training.

1.2 Project overview

Safyron Learning is a compliance-oriented learning management platform developed as a full-stack web application. Its visual identity is based on Safyron Engineering and its functional vocabulary reflects industrial training: mandatory modules, published resources, assessment attempts, audit records and instructor approval.

The project combines a responsive Angular single-page application with a Spring Boot REST backend. MySQL stores users, modules, materials, quizzes, questions, learner progress and quiz results. Uploaded PDF, DOCX and PPTX documents are retained as learning resources and can also provide the source text for quiz generation.

1.3 Initial problem

Safyron Engineering's technical activities depend on documents, procedures and regulatory knowledge that must remain available to employees. Before the platform, training material, quiz preparation and evidence of completion could be handled as separate activities. This separation made it difficult to know which version of a module was current, whether a learner had completed the assessment and whether the result had been reviewed by a responsible instructor.

The project therefore models the training process as a complete lifecycle: material is attached to a module, an assessment is prepared, the module is published, the learner submits answers once, and the instructor records the final decision. The administrator receives a transverse view for governance and data maintenance.

Traditional training workflows often separate the content repository, questionnaire authoring and evidence of completion. This creates several problems:

- learners cannot easily identify the current published material;
- instructors spend significant time rewriting questions from documents;
- a score alone does not provide enough evidence for approval;
- repeated attempts can invalidate the meaning of an assessment;
- deleting a module or user can fail because of relational dependencies.

The project question was therefore: *how can a web platform connect learning resources, AI-assisted assessment and instructor validation while preserving data consistency?*

1.4 Stakeholders

1.4.1 Learner

The learner browses published modules, opens or downloads files, takes an available quiz and sees the submission state. The score is not exposed as a learner-facing decision; certification is controlled by the instructor.

1.4.2 Instructor

The instructor creates and manages modules, imports source material, produces AI-generated or manual quizzes, publishes assessments and reviews detailed answers. Approval can be removed if it was granted accidentally.

1.4.3 Administrator

The administrator manages users, modules, quizzes and operational statistics. The dashboard groups submissions by module and provides a global governance view.

1.5 Scope and objectives

Table 1.1: Project objectives

Objective	Expected result
Module management	Create, publish, update and delete modules with dependencies.
Learning resources	Upload, preview, open and download training files.
Assessment	Generate quizzes with AI or create them manually.
Controlled attempt	Store one submission per learner and quiz.
Human validation	Display detailed answers and support approval or reversal.
Administration	Manage users, modules and grouped submission indicators.
Quality	Validate the API through tests, builds and Swagger.

1.6 Planning

The month was divided into four iterations. The first week focused on requirements, architecture and the existing codebase. The second covered modules, files and learner navigation. The third implemented quiz generation, manual authoring, one-time submission and review. The fourth concentrated on administration, deletion consistency, Swagger, interface improvements and end-to-end validation.

Table 1.2: Simplified Gantt diagram – provisional internship planning

Proposed activity	**S1** 01–07	**S2** 08–14	**S3** 15–21	**S4** 22–31
Understanding the organisational context	■			
Requirements analysis and scope definition	■			
Functional analysis and UML modelling		■		
Architecture and data-model design		■		
Backend implementation			■	
Progressive integration and finalisation			■	■
Testing and validation				■
AI integration and final corrections				■

1.7 Project boundaries

The project does not attempt to provide payments, video conferencing, multi-tenant billing or a complete certificate document generator. These features are suitable future extensions. The internship focused on a reliable learning and assessment core.

Conclusion

The company context justifies a platform focused on traceable learning and controlled assessment rather than a generic document repository. The next chapter translates this context into functional and non-functional requirements, architecture decisions and a persistent data model.

Chapter 2

Analysis and Design

Introduction

The design phase transforms the company context into explicit requirements, responsibilities, workflows and data structures. The following sections separate what the platform must do from how the Angular, Spring Boot and MySQL components implement it.

2.1 Functional requirements

The platform is organized around a module lifecycle. An instructor creates a module, attaches source documents, optionally generates a quiz, reviews it and publishes the module. A learner can then consult the published material and submit the associated quiz once. The result remains pending until the instructor reviews the answers.

Table 2.1: Functional requirements matrix

ID	Function	Acceptance criterion
F01	Authentication	An existing user can access the interface according to the role.
F02	Module	A module has a title, description, category, priority and state.
F03	Material	A file is attached to one module and can be opened or downloaded.
F04	AI generation	A source document produces an editable quiz draft.
F05	Manual quiz	An instructor can create questions and answer options directly.
F06	Publication	Only a reviewed quiz can be made available to learners.
F07	Submission	A learner submits a quiz only once.
F08	Review	The instructor sees the learner, module and answer details.
F09	Approval	Approval can be granted and later removed.
F10	Administration	Users, modules and grouped submissions are visible to admins.

2.2 Actors and principal use cases

Table 2.2: Actors and responsibilities

Actor	Main responsibilities
Learner	Consult published material, submit the associated quiz once and follow the approval state.
Instructor	Create and publish modules, author quizzes, review detailed answers and approve or revoke submissions.
Administrator	Manage users, modules and quizzes, and consult grouped operational activity.

2.2.1 Non-functional requirements

In addition to functional behavior, the platform must satisfy the following quality requirements:

- **Security:** access is authenticated with JWT and controlled by role-based permissions.
- **Consistency:** module deletion removes dependent quizzes, materials, progress and results in a controlled transaction.
- **Traceability:** learner answers, submission dates and instructor approval decisions are persisted.
- **Maintainability:** controllers, services, repositories and integration clients have separate responsibilities.
- **Usability:** each role can identify its next action quickly on a responsive interface.
- **AI continuity:** manual quiz authoring remains available when the external AI provider is unavailable.
- **Availability:** learning material remains accessible after a quiz has been submitted.

Table 2.3: Non-functional requirements and acceptance evidence

Requirement	Acceptance evidence
Security	JWT authentication and role-aware API access.
Consistency	Transactional module and user deletion workflows.
Traceability	Stored answer map, date, learner, quiz and approval state.
Maintainability	Layered services, repositories and specialized integrations.
Usability	Role-specific dashboards and disabled submitted-quiz action.
AI continuity	Manual quiz creation and explicit AI error responses.

2.3 Use-case model

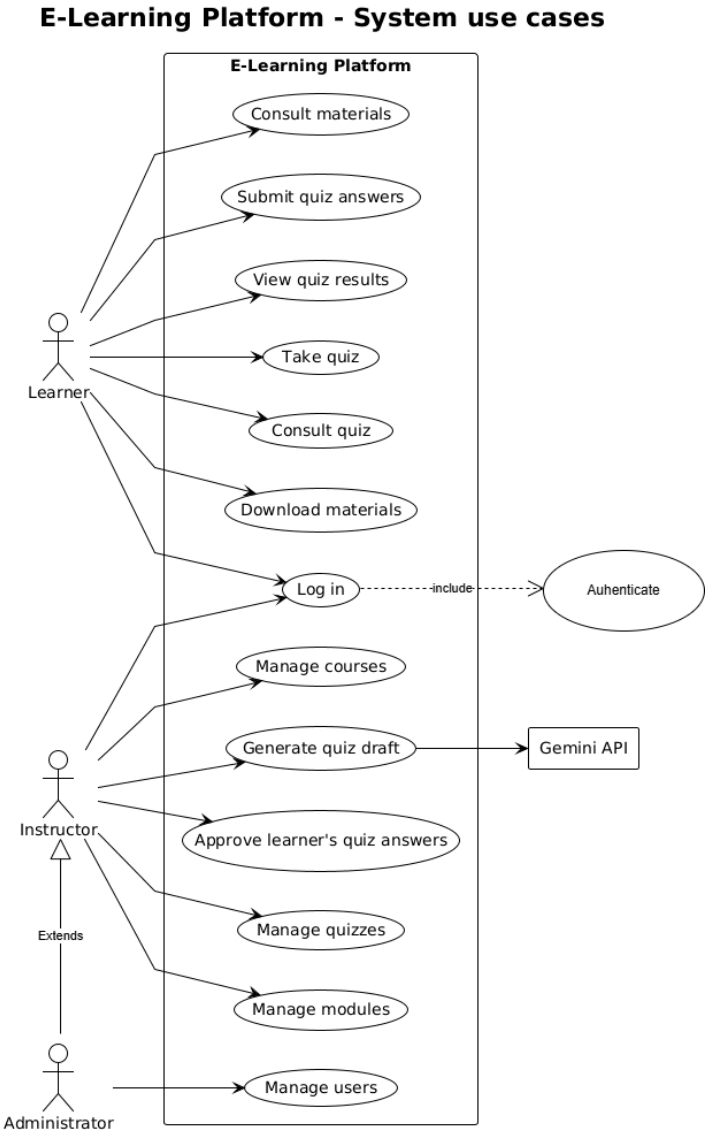


Figure 2.1: Global use-case diagram

The use-case model separates responsibilities while keeping the main training workflow shared. AI generation is an assistance function, not an automatic certification function: generated content must remain editable and subject to human review.

2.3.1 Module and material scenario

The instructor creates a module with a title, description, category and priority, then attaches one or more PDF, DOCX or PPTX files. The files remain associated with the module and can be opened or downloaded by a learner once the module is published. Deleting the module removes its dependent materials.

2.3.2 Quiz authoring and publication scenario

The instructor chooses either manual authoring or AI generation. In the AI case, extracted document text is sent to the provider and the returned draft is validated before it is stored. In the manual case, the instructor enters questions, options and correct answers directly. In both cases, publication is a separate action so that an unreviewed draft is not exposed to learners.

2.3.3 Learner submission scenario

The learner opens the published module, consults its material and selects one option per question. The answer map is stored with persistent question identifiers. The first submission creates a result and sets the assessment progress to 100 percent; a second submission is rejected and the material remains available.

2.3.4 Instructor approval scenario

The instructor sees a compact submission row containing the learner, module, quiz and date. A details action expands the question-by-question answers and correct answers. The instructor can approve the result or remove the approval after a mistake. Certification is therefore a human decision rather than a score-only calculation.

2.3.5 Textual description of major use cases

Table 2.4: Main use-case scenarios

Use case	Nominal and alternative scenarios
Publish module	The instructor creates a module, attaches material and publishes it. If required information is missing or ownership is invalid, publication is rejected.
Generate quiz	The instructor selects material and requests a draft. If extraction or the external API fails, an explicit error is returned and manual authoring remains available.
Submit quiz	The learner answers and submits once. If a result already exists, the second submission is rejected and the material remains accessible.
Review submission	The instructor opens a submission and expands the detailed answers. Approval can be granted or revoked after review.

2.4 User-interface conception

The learner dashboard prioritizes published modules, accessible files and quiz availability. The instructor dashboard prioritizes creation, publication and answer review. The administrator dashboard prioritizes counts, grouped submissions and management actions. These role-oriented interface decisions are implemented later in Chapter 3; the screenshots there are validation evidence rather than requirements analysis.

2.5 System architecture

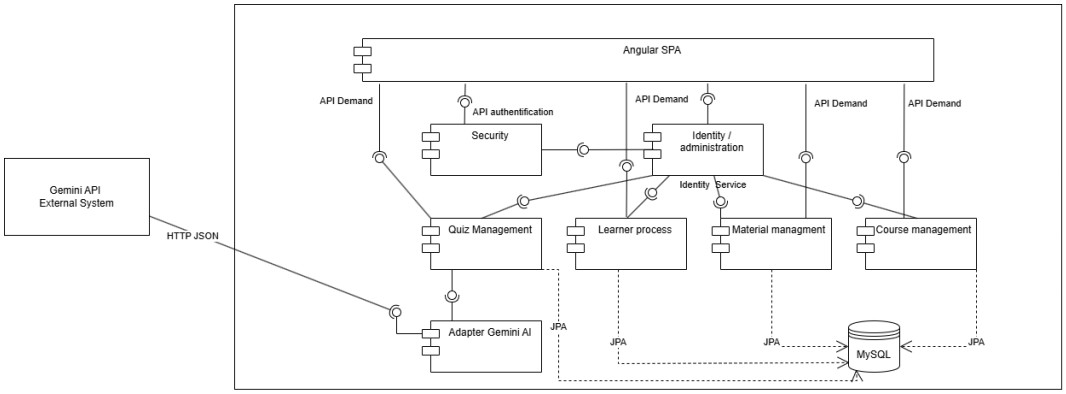


Figure 2.2: Component architecture

The Angular client communicates with REST controllers. Controllers delegate to services, services coordinate repositories and domain entities, and the document extraction and AI clients are isolated behind dedicated components. This separation prevents UI concerns from leaking into persistence logic.

2.6 Technical conception

The technical conception uses a layered REST architecture. The controller receives the request, the service applies the business rule, the repository persists the entity and the database stores the durable state. The external AI API is an integration boundary and is not part of the core database model.

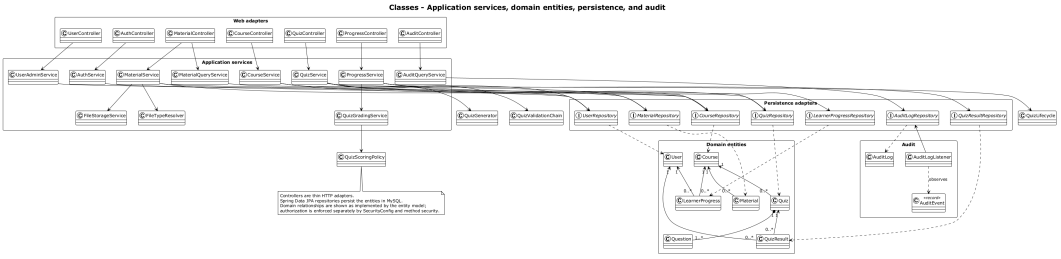


Figure 2.3: Entity–repository–service responsibilities

2.7 Design principles and patterns

The solution follows a layered architecture and the Single Responsibility Principle. Controllers map HTTP requests, services implement business rules, repositories perform persistence queries and entities represent the relational model. This separation makes the deletion and submission rules testable without coupling them to Angular templates.

Document processing follows a strategy-like organization: the extraction responsibility is isolated from quiz generation so that PDF, DOCX and PPTX support can evolve independently. The quiz lifecycle is treated as a state transition problem, separating draft, published and submitted behavior. Dependency inversion is applied to the external AI provider through a dedicated client boundary, allowing tests to replace the remote service with a stub.

2.8 Security design

JWT authentication is integrated as the application authentication mechanism. After a successful login, the server issues a signed token containing the identity and role claims. The Angular client attaches the bearer token to API requests, while Spring Security validates the signature and protects role-specific routes. This design keeps authorization at the API boundary rather than relying only on hidden buttons in the interface.

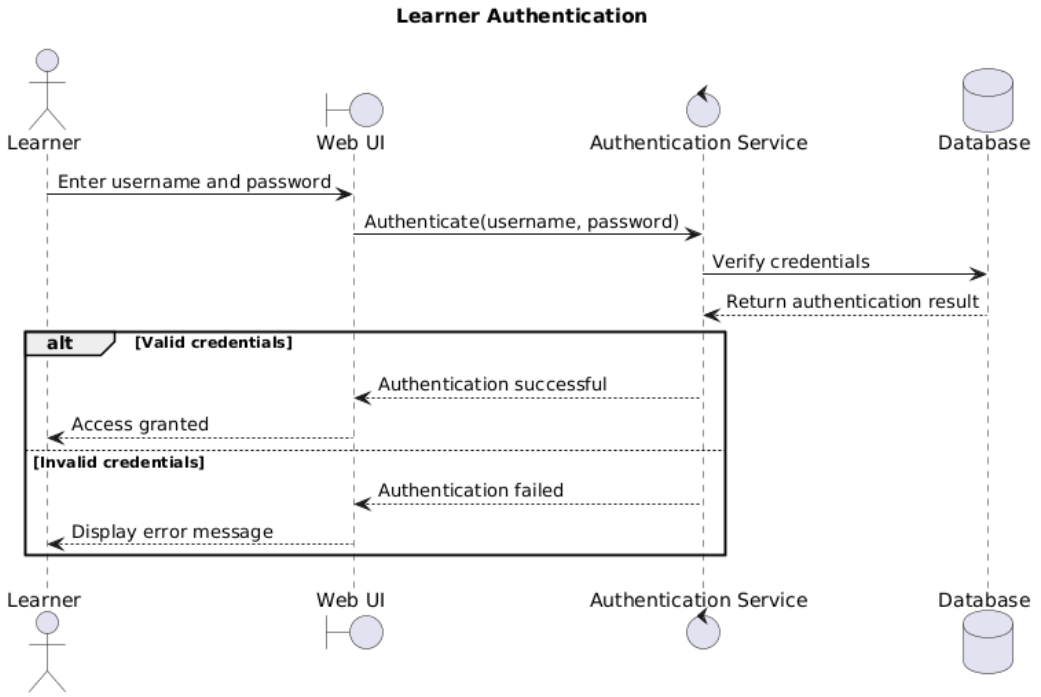


Figure 2.4: Authentication sequence

Table 2.5: Security responsibilities

Component	Responsibility
Login service	Verifies credentials and issues a signed JWT.
Angular interceptor	Adds the bearer token to API requests.
Security filter	Validates signature, expiration and claims.
Role mapping	Converts claims into learner, instructor or admin authorities.
Protected controller	Rejects unauthorized or forbidden operations.
Password storage	Stores a one-way password hash, never the clear password.

2.9 Data model and integrity

The main relationships are User–Course, Course–Material, Course–Quiz, Quiz–Question and Learner–QuizResult. A quiz belongs to its module so that the module is the lifecycle owner. Deletion is transactional: results, progress records, quizzes and materials are removed in a

controlled order before the module itself is deleted. User deletion removes learner evidence and detaches owned resources where required.

Table 2.6: Persistent entity dictionary

Entity	Main fields	Responsibility
User	id, name, email, role	Learner, instructor and administrator identity.
Course	id, title, status, instructor	Training module and lifecycle owner.
Material	id, file name, course	Uploaded PDF, DOCX or PPTX resource.
Quiz	id, title, course, published	Assessment attached to one module.
Question	id, text, options, correct answer	Question content and grading key.
QuizResult	learner, quiz, answers, approved	Submission evidence and approval state.
Learner progress	learner, course, progress	Course completion tracking.

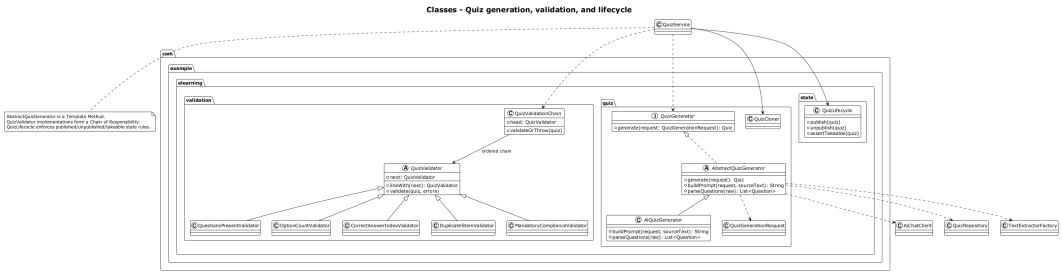


Figure 2.5: Quiz validation and lifecycle classes

2.10 Quiz lifecycle

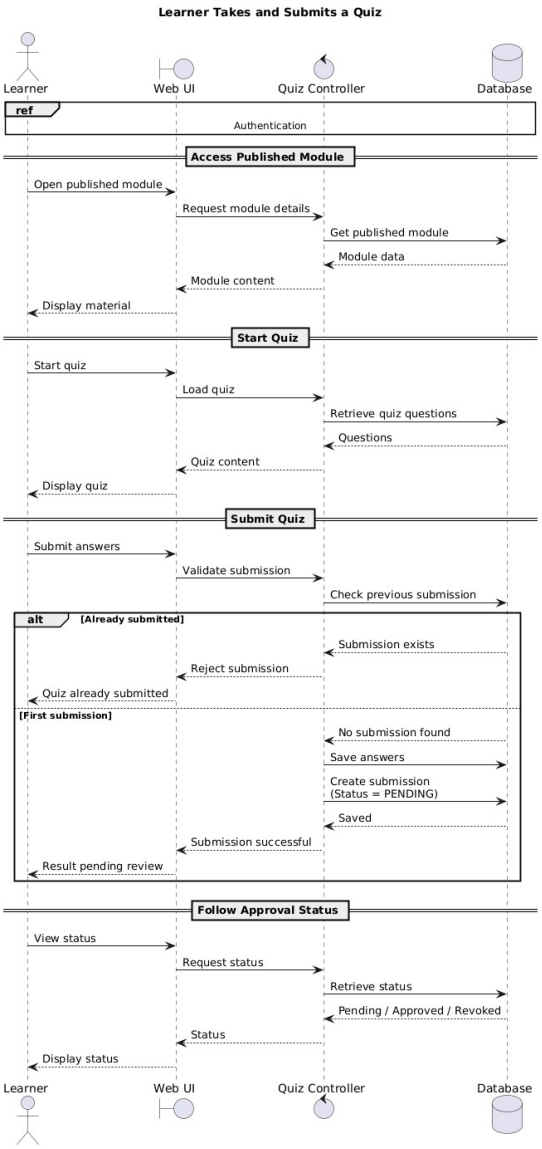


Figure 2.6: Learner takes and submits a quiz

The lifecycle distinguishes draft, published and submitted states. A learner can continue to consult learning material after submission, but the quiz action becomes unavailable. The approval state is separate from the score: submission means evidence was received, while approval means an instructor has reviewed the evidence.

2.11 AI design

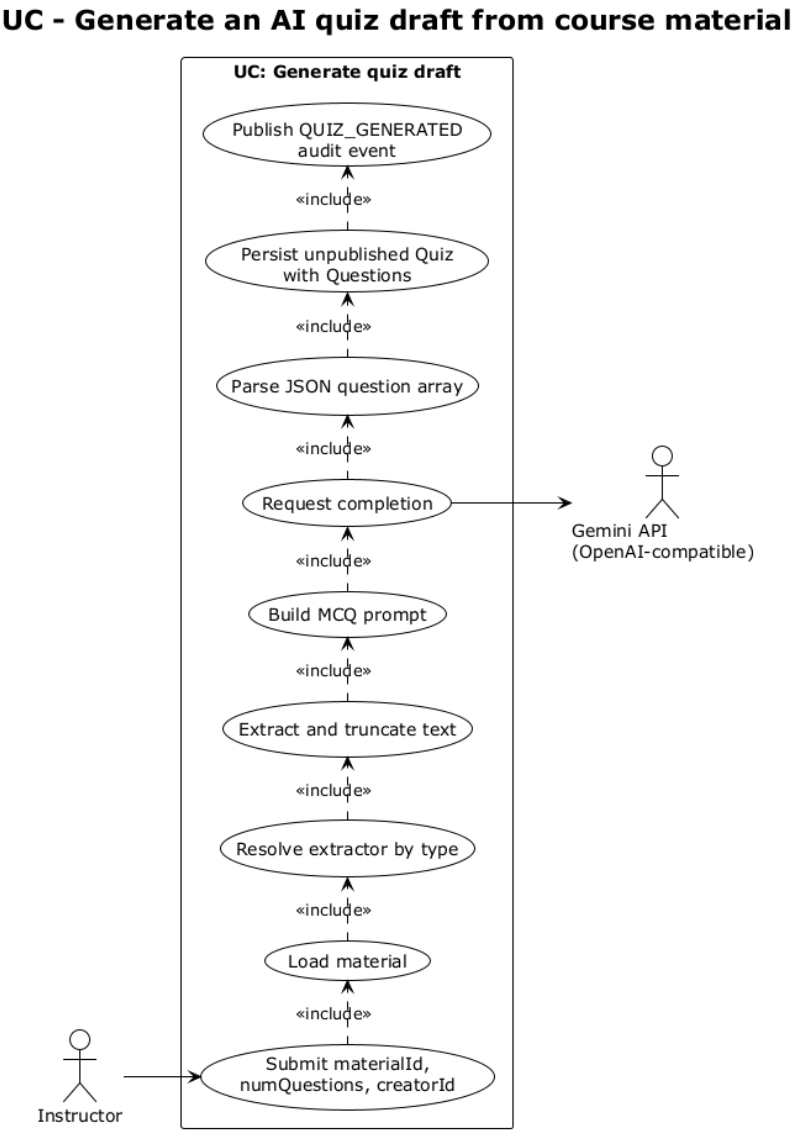


Figure 2.7: Detailed AI quiz-generation use case

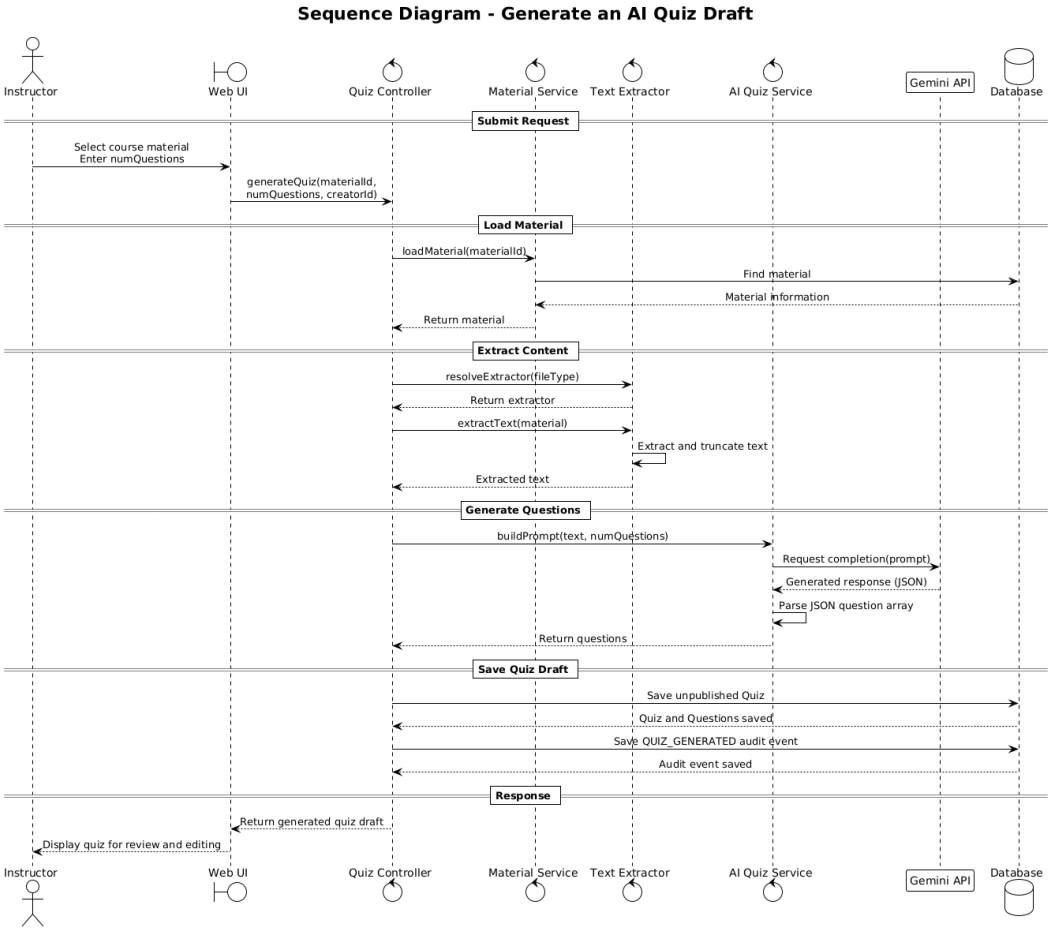


Figure 2.8: AI quiz-generation sequence

The sequence diagram follows the requested analysis notation: the user initiates the action through the Angular frontend, the controller reads the material from the database, and the controller calls the external API. The returned draft is validated and stored as an unpublished quiz. Invalid or empty responses are reported explicitly and never bypass instructor publication.

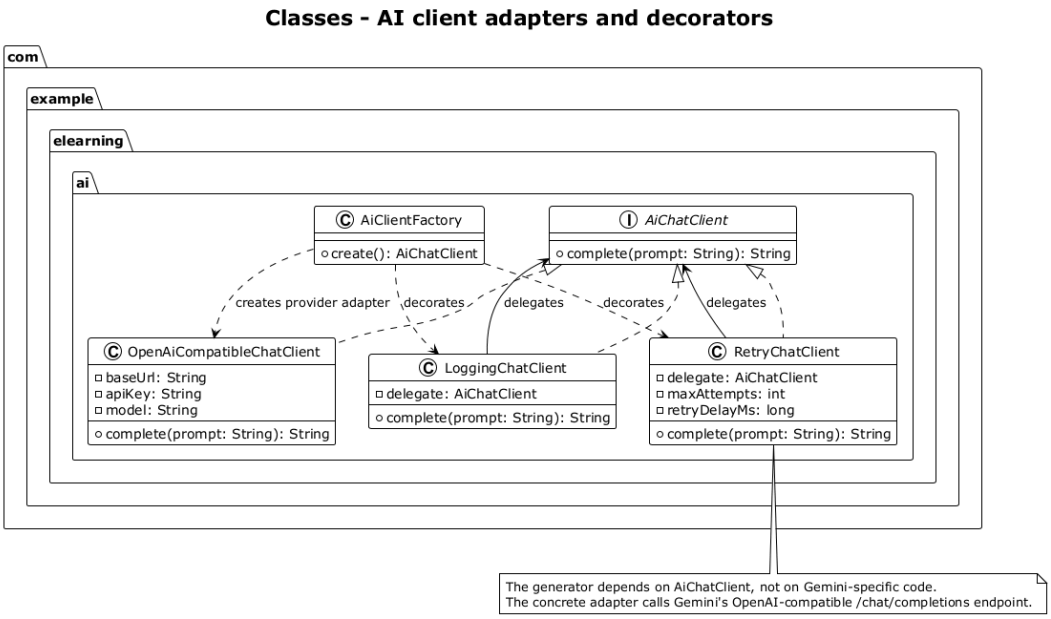


Figure 2.9: AI client and generation classes

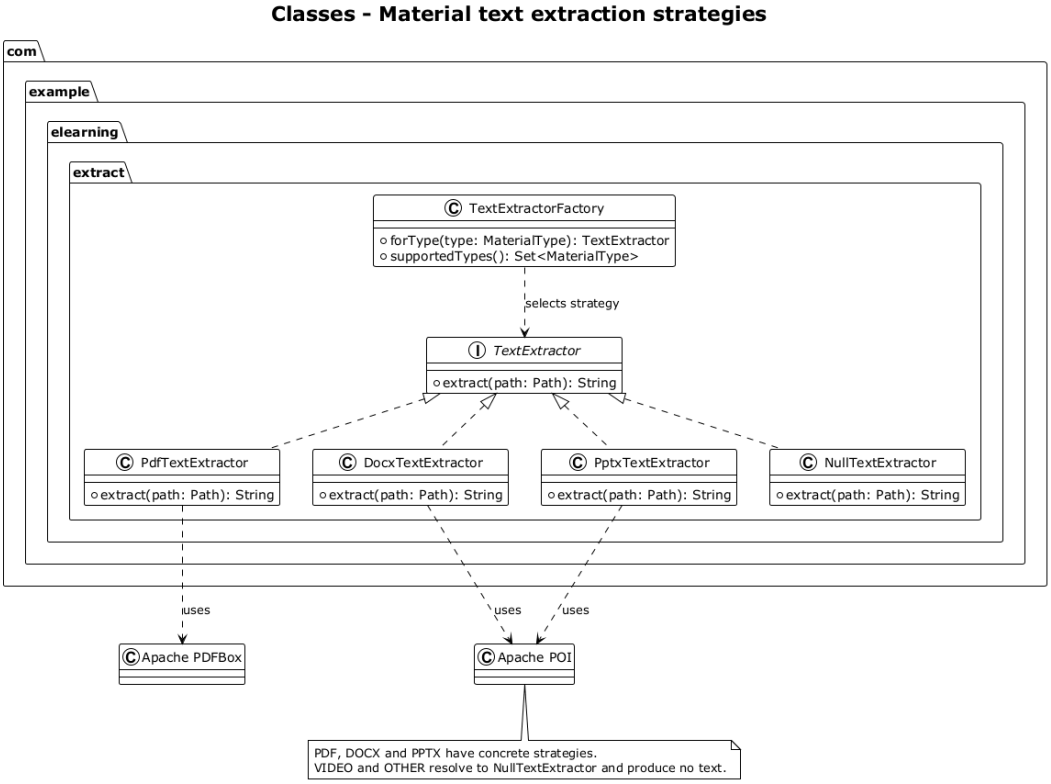


Figure 2.10: Document extraction classes

Conclusion

The analysis establishes a traceable relationship between company needs, quality requirements, security, persistence and implementation boundaries. The next chapter describes how these decisions were realized and validated in the backend, frontend, dashboards and API workflows.

Chapter 3

Implementation and Validation

Introduction

This chapter presents the implementation of the principal workflows and the technical environment used during the internship. It connects the design decisions to concrete backend services, Angular screens, persistence rules and API operations.

3.1 Development environment and technologies

Table 3.1: Technologies and versions

Technology	Version	Purpose
Angular	21	Single-page frontend application.
TypeScript	5.9	Frontend implementation language.
Spring Boot	3.3.5	REST backend framework.
Java	21 target	Backend implementation language.
Spring Data JPA	Project version	Relational persistence and repositories.
MySQL	Project environment	Main relational database.
H2	Test environment	Isolated backend test database.
Maven	Project environment	Backend dependency and build management.
Node.js / npm	Project environment	Frontend dependency and build management.
Gemini API	Gemini Flash	AI-assisted quiz generation.
OpenAPI / Swagger	Project dependency	Interactive API documentation and testing.

3.2 Backend implementation

The backend is implemented with Spring Boot 3.3, Spring Web, Spring Data JPA, validation and Spring Security. Controllers expose focused REST resources for authentication, courses, materials, quizzes, progress and administration. Services contain the business rules and repositories isolate database access.

Table 3.2: Main backend layers

Layer	Responsibility
Controller	HTTP mapping, request validation and response status.
Service	Lifecycle rules, ownership, grading and transactional operations.
Repository	JPA queries and dependent-record lookup.
Entity	Relational domain model and associations.
Integration	Gemini client, document extraction and file serving.

3.3 Frontend implementation

The frontend is an Angular 21 single-page application styled with Tailwind utilities and a shared Safyron design system. Shared navigation, cards, status badges, tables and action controls are reused across role dashboards. The interface deliberately exposes different actions to learners, instructors and administrators.

3.4 Learner experience

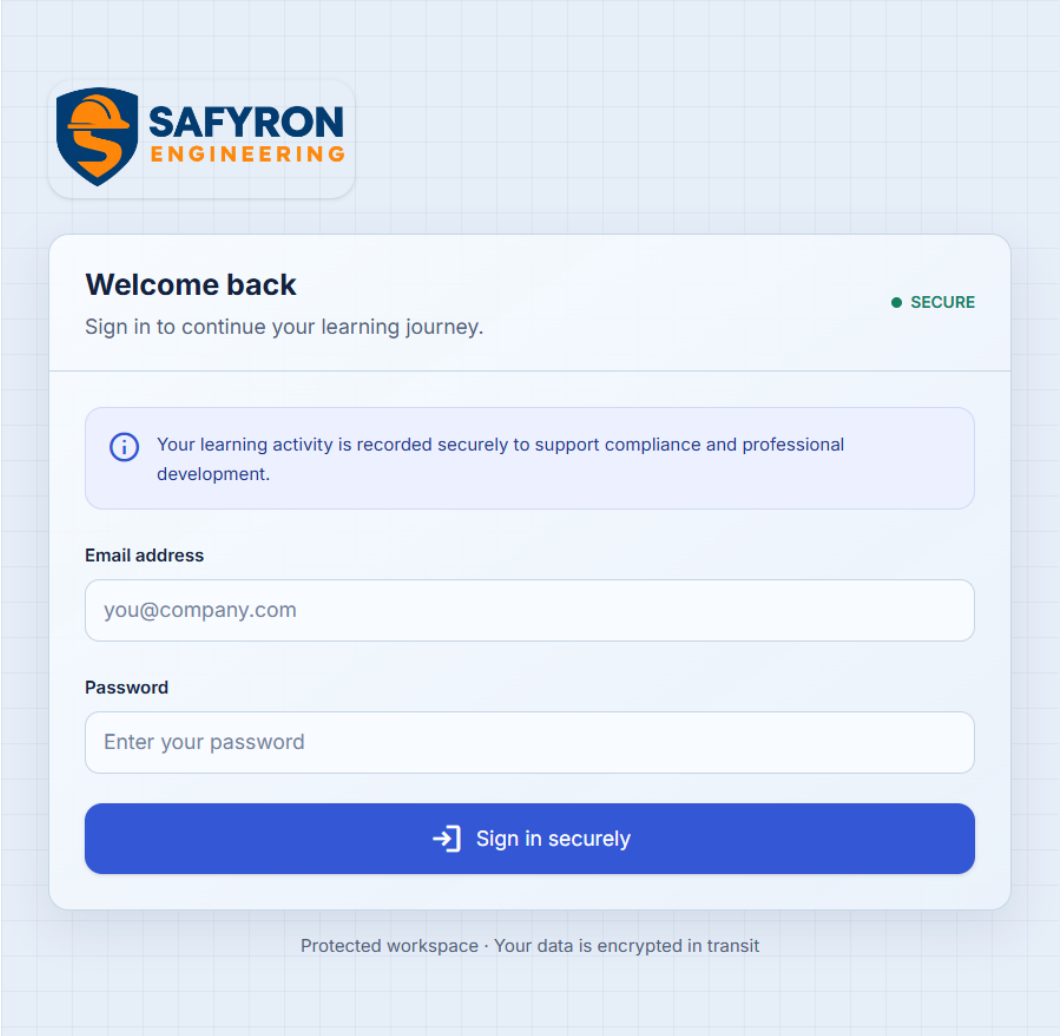


Figure 3.1: Live screenshot captured from the running login page

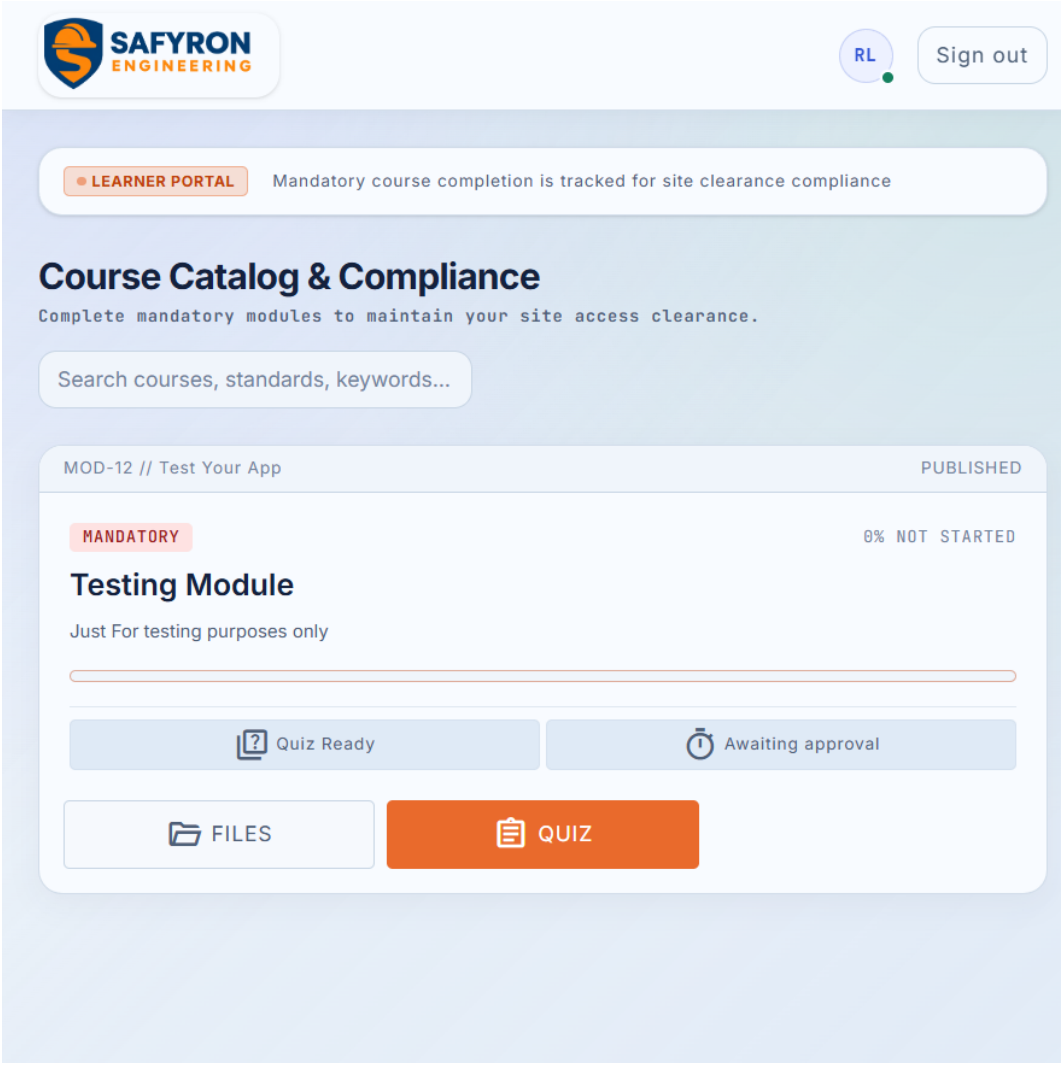


Figure 3.2: Live screenshot captured from the learner portal

The learner portal presents published modules with their completion state, material controls and quiz availability. Files can be opened in the browser or downloaded. Once a quiz has been submitted, its action is visually disabled while the learning material remains available.

3.5 Instructor studio

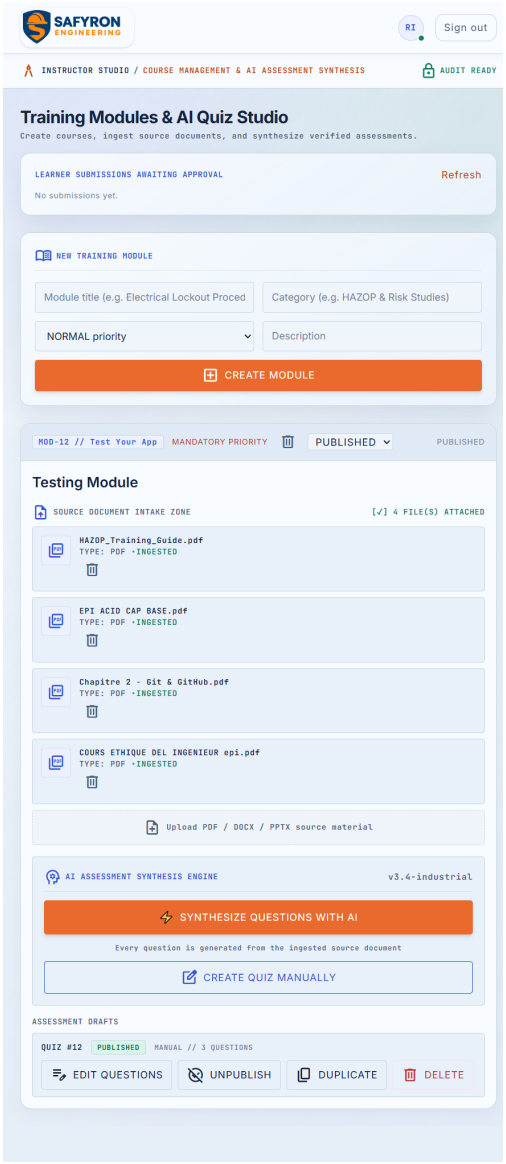


Figure 3.3: Live screenshot captured from the instructor studio

The instructor studio combines module management, document intake, AI quiz synthesis and manual authoring. It also presents pending submissions with learner and module context. The detailed view expands the selected submission to show each question, the learner answer and the correct answer.

3.6 Administrator console

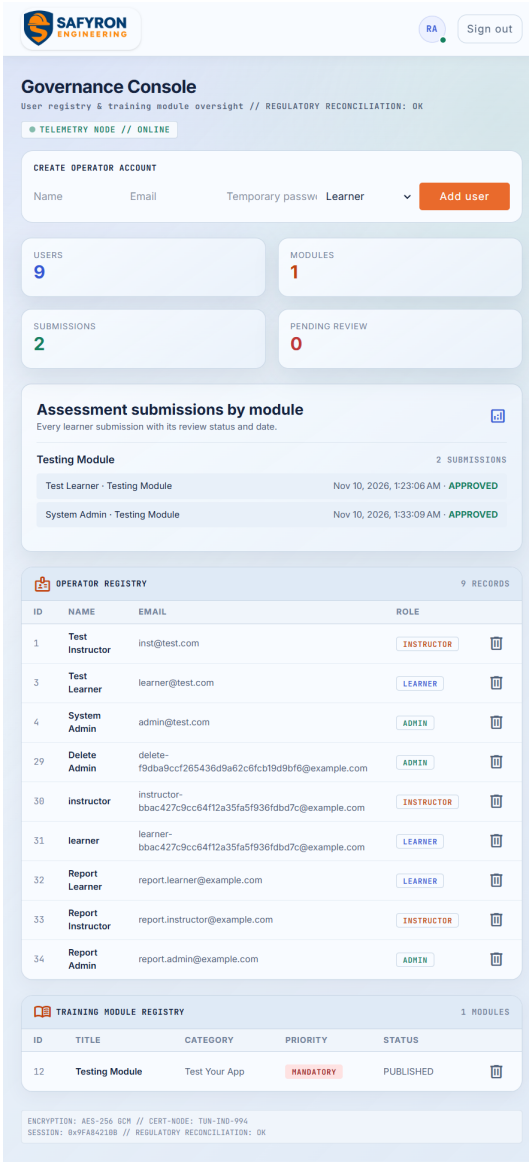


Figure 3.4: Live screenshot captured from the administrator console

The administrator console provides user creation and deletion, module oversight, quiz management and submission indicators. The grouped view makes the module the primary reporting unit and includes the submission date and review state.

3.7 Answer persistence and grading

Answers are stored using the persistent question identifier rather than the position of a question in the displayed array. This distinction prevents review screens from displaying “Not answered” when the learner selected option zero or when questions were reordered. The grading service stores the submitted answer map as JSON and creates one result per learner and quiz.

3.8 File management and deletion

The material controller exposes an inline file endpoint and a download action. Course deletion is transactional and removes quiz results, progress, quizzes and materials before deleting the module. User deletion similarly removes learner-dependent records and detaches owned entities. These measures prevent the foreign-key error that originally appeared when deleting users directly from the database.

3.9 Swagger and API usage

OpenAPI documentation is exposed by the backend and provides an interactive Swagger UI for testing. Typical operations include:

```
POST /api/auth/login
GET  /api/courses/published
POST /api/quizzes/generate
POST /api/quizzes/manual
POST /api/progress/quiz/{quizId}/submit
PUT  /api/quizzes/results/{resultId}/approve
PUT  /api/quizzes/results/{resultId}/unapprove
```

3.10 Implementation difficulties and corrective actions

Several defects were discovered through integration and browser validation. Learner answers initially used array positions while the instructor review expected persistent question identifiers; the answer binding was corrected and legacy values were normalized. A second quiz attempt was initially possible, so the service now rejects an existing learner–quiz result with HTTP 409.

Module and user deletion initially failed because dependent rows remained in the database. Transactional cleanup, repository deletion methods and database-level cascade or “SET NULL” behavior were added. AI generation also produced misleading forbidden responses when an application exception was forwarded to a protected error route; explicit error responses now expose the actual failure to the client.

3.11 Validation and Results

Validation introduction

Validation was performed at several levels because a successful compilation does not prove that the learning lifecycle is correct. The checks below cover automated backend tests, frontend builds, API behavior, database consistency and browser journeys for the three roles.

3.12 Validation strategy

Validation combined unit and integration tests for the backend, an Angular production build, Swagger checks and live HTTP scenarios. The objective was not only to verify that an endpoint returned a response, but to verify the business rule represented by that response.

Table 3.3: Validation activities

Activity	Result
Backend clean tests	Passed after CRUD, grading and deletion changes.
Angular production build	Passed with the final dashboard and branding changes.
OpenAPI availability	Swagger UI and OpenAPI JSON returned successfully.
Module deletion	Returned successful deletion after dependent cleanup.
Answer persistence	Review returned the selected option for the correct question.
Duplicate submission	A second attempt was rejected with HTTP 409.
User deletion	Dependent learner records were removed without a foreign-key failure.

3.13 Validation traceability

Table 3.4: Traceability between requirements and validation

Requirement	Validation evidence
Security	JWT-aware login design, role-aware routes and protected operations.
Consistency	Real module and user deletion scenarios without foreign-key failure.
Traceability	Detailed answer review, submission date and approval/unapproval.
Maintainability	Successful layered backend tests and Angular production build.
AI continuity	Manual quiz creation remains available beside AI generation.
One-time assessment	First submission succeeds and repeated submission returns 409.
Material availability	Files remain accessible after quiz submission.

3.14 Functional scenarios

The first scenario creates a module and attaches documents. The second generates or manually creates a quiz and publishes it. The learner then opens the module, selects answers and submits once. The instructor sees the submission in the review area, expands the detailed answers and approves it. The administrator can consult the grouped submission statistics.

3.15 Quality results

The principal defects discovered during development were incorrect answer keys, misleading HTTP 403 responses caused by error forwarding, incomplete dependent deletion and an interface that allowed a second attempt. Each issue was fixed at its root: question IDs are now used

for answer maps, application errors return explicit statuses, deletion is transactional, and both backend and frontend prevent retakes.

3.16 Security and operational limitations

The JWT integration provides the intended authentication and authorization boundary for deployment. Secret values, including the AI provider key, must remain in environment configuration and must be rotated if exposed. File uploads should receive size, type and content scanning limits before production. Rate limiting and audit retention are also recommended for a public deployment.

3.17 Difficulties, limitations and perspectives

The most important difficulties were not isolated coding errors but interactions between features. Deleting a module required understanding all dependent records. Persisting an answer required a stable question key rather than a temporary array index. The approval workflow required a reversible state, not only a one-way button. These issues were corrected through service rules, repository operations and end-to-end checks.

The current validation does not constitute a production load test or an independent security audit. Future work should measure performance with a larger catalogue, add automated accessibility checks, scan uploaded files, rotate refresh tokens, export audit records and add notifications. The AI workflow should also display confidence or source references and be evaluated against a larger curated question corpus.

3.18 User experience observations

The live screenshots show a coherent visual language: blue-gray surfaces, strong contrast, shield branding, status badges and action-oriented cards. The instructor screen emphasizes operational work, while the learner screen keeps the path to course material and assessment simple. The administrator screen prioritizes counts, grouped submissions and governance actions.

3.19 Personal contribution

This internship required work across requirements analysis, Angular components, Spring services, relational integrity, third-party AI integration, API documentation and testing. The most valuable lesson was that a feature is not complete when its happy path works: its deletion behavior, error response, role boundary and persistence format must also be verified.

Conclusion

The implementation connects the role-specific Angular experience to a transactional Spring Boot backend, and the validation activities confirm the main learner, instructor and administrator workflows. The report also identifies the remaining production-hardening work without overstating the evidence obtained during a one-month internship.

General Conclusion

The Safyron Learning project delivered a coherent foundation for compliance training. It connects documents, modules, AI-assisted quiz authoring, learner submissions and instructor approval in one workflow. The architecture separates the Angular interface, Spring Boot services, persistence and external integrations, which makes the solution easier to evolve.

The implementation also addressed the less visible aspects of quality: one-time assessment attempts, question-accurate answer persistence, transactional deletion, explicit API errors and a professional administrative view. JWT authentication is part of the security design and provides the appropriate basis for role-aware deployment.

Future work should include certificate generation, richer audit exports, notifications, document versioning, stronger file scanning, refresh-token rotation, automated accessibility checks and performance testing with a larger dataset. The AI workflow could also gain confidence indicators and a teacher feedback loop. These extensions would transform the current internship deliverable into a production-scale learning service.

References and Webography

- Spring Boot Reference Documentation, https://docs.spring.io/spring-boot/docs/current/reference/html/.
- Angular Documentation, https://angular.dev/.
- Spring Security Reference, https://docs.spring.io/spring-security/reference/.
- OpenAPI Specification, https://spec.openapis.org/oas/latest.html.
- Google Gemini API Documentation, https://ai.google.dev/gemini-api/docs.
- Jakarta Persistence Specification, https://jakarta.ee/specifications/persistence/.
- OWASP Application Security Verification Standard, https://owasp.org/www-project-application-security-verification-standard/.

Appendix A

Project Structure and Reproduction

A.1 Directory structure

```
report/
  report.tex
  chapters/
    introduction.tex
    chapter1-context.tex
    chapter2-design.tex
    chapter3-implementation-validation.tex
    conclusion.tex
    references.tex
    appendix.tex
  assets/
    approved-diagrams/
    screenshots/
```

A.2 Representative REST contracts

```
POST /api/quizzes/manual
{
  "courseId": 11,
  "title": "Safety assessment",
  "questions": [
    {
      "text": "Which action is required before maintenance?",
      "options": ["Lockout", "Ignore", "Delay", "None"],
      "correctAnswer": 0
    }
  ]
}

POST /api/progress/quiz/13/submit?learnerId=5
{"50": 1}

PUT /api/quizzes/results/21/approve
PUT /api/quizzes/results/21/unapprove
```

A.3 Diagram catalogue

Table A.1: Project diagrams stored with the report

Reference	Description
01	Global use cases.
02–03	AI generation use case and sequence.
04	Authentication sequence.
05a–05d	AI, extraction, validation and service classes.
06	Component architecture.
07	Material activity diagram.
08	Learner quiz-taking and submission sequence.